UNIVERSITÀ
DEGLI STUDI
DI BERGAMO

Dipartimento di
Ingegneria Gestionale, dell'Informazione e della Produzione

Corso di Laurea in
Ingegneria Informatica

Progettazione algoritmi e computabilità

Giorgio Passarella
Matricola n. 1079287

Davide Brambilla
Matricola n. 1080752

Massimiliano Federico Gervasoni
Matricola n. 1069211

Anno Accademico
2025/2026

Indice

Elenco delle figure **iii**

1 Iterazione 0 **1**

1.1 Introduzione 1

1.2 Requisiti 3

1.3 Casi D'uso 5

1.4 Deployment Diagram 7

1.5 Architettura del Sistema 8

Elenco delle figure

1.1 Casi d'uso 5
1.2 Diagramma di deployment 7
1.3 Archiettura MVC del Sistema 9

Capitolo 1

Iterazione 0

1.1 Introduzione

OptiTour è un'app che ottimizza gli itinerari turistici calcolando automaticamente il percorso più efficiente per visitare una serie di luoghi nel tempo a disposizione.
Scegliere cosa visitare è semplice, ma organizzare le tappe nell'ordine migliore richiede tempo, esperienza e una valutazione accurata dei tempi di spostamento e di visita. A differenza delle comuni app di mappe, OptiTour non si limita a mostrare i punti di interesse, ma trasforma una lista di monumenti in un itinerario ottimale.
L'utente seleziona il punto di partenza e i luoghi che desidera visitare, indicando quanto tempo ha a disposizione. L'app, elaborando queste informazioni, dà origine al percorso più rapido, garantendo che l'itinerario rispetti la finestra temporale impostata.

Con OptiTour, l'utente può:

- **Pianificazione Itinerari Smart:** selezione dei punti di interesse (POI) e calcolo del ciclo ottimale per ridurre chilometri e tempi di percorrenza.

- **Gestione Flessibile:** possibilità di creare nuovi viaggi (manuali o casuali), modificare le tappe, visualizzare dettagli o eliminare percorsi.

- **Social e Storico:** salvataggio dei percorsi preferiti, consultazione dello storico dei viaggi effettuati, condivisione e aggiunta di recensioni.

- **Gestione Profilo:** registrazione, login e gestione sicura dei dati personali e delle credenziali.

L'applicazione si interfaccia direttamente con provider di servizi geografici esterni per garantire che il calcolo delle distanze e dei tempi sia basato su dati reali e aggiornati.

Perché scegliere OptiTour?

OptiTour nasce per rispondere a un'esigenza specifica: l'efficienza nella programmazione di itinerari. Spesso i turisti perdono tempo prezioso camminando avanti e indietro per la città a causa di una pianificazione non ottimale o assente. Questo problema, noto come *Traveling Salesman Problem* (TSP), diventa difficile da risolvere mentalmente man mano che il numero di luoghi da visitare aumenta.
OptiTour automatizza questo processo complesso utilizzando algoritmi avanzati sui grafi ed euristiche dedicate. Il sistema non offre solo un percorso, ma il miglior percorso possibile, rendendolo lo strumento ideale per diverse tipologie di utenti: dal turista che vuole massimizzare le visite nel tempo a disposizione, allo sportivo che cerca un percorso di allenamento circolare che tocchi specifici punti di interesse, fino ai professionisti dell'ospitalità che desiderano offrire itinerari personalizzati ai propri ospiti.
Con un'interfaccia intuitiva che separa nettamente la logica di presentazione da quella algoritmica, OptiTour offre una soluzione potente ma accessibile per trasformare il modo in cui esploriamo le città.

1.2 Requisiti

Il sistema deve soddisfare i seguenti requisiti funzionali e non funzionali per garantire un'esperienza di pianificazione ottimale:

Requisiti funzionali

- **Gestione autenticazione e profilo:** gli utenti devono poter effettuare la registrazione, il login e il logout. Il sistema deve permettere la gestione del profilo includendo la modifica dei dati personali, il cambio password e l'eliminazione definitiva dell'account.

- **Visualizzazione e catalogo:** il sistema deve offrire una vista astratta per la visualizzazione dei percorsi, permettendo all'utente di sfogliare un catalogo, applicare filtri di ricerca ed esportare l'itinerario desiderato.

- **Creazione itinerario manuale:** l'utente può generare un nuovo viaggio impostando un punto di partenza e selezionando manualmente i monumenti e le tappe d'interesse.

- **Creazione itinerario casuale:** generazione automatica basata su percorsi esistenti o nuovi, con la possibilità di definire tempistiche specifiche per la visita.

- **Ottimizzazione del percorso (TSP):** il sistema deve integrare un modulo di ottimizzazione che, interfacciandosi con un Provider di servizi geografici esterno, calcoli il percorso ottimo. Tale ottimizzazione è inclusa automaticamente durante la creazione o la modifica delle tappe.

- **Gestione e stato viaggi:** gli utenti devono poter salvare percorsi, modificarne le tappe, eliminarli o contrassegnarli come "completati".

- **Social e storico:** il sistema deve gestire uno storico dei viaggi che permetta la condivisione degli itinerari e l'aggiunta di recensioni per i percorsi effettuati.

Requisiti non funzionali

- **Architettura modulare:** il sistema deve separare nettamente le funzionalità di interfaccia (vista utente) dalle logiche di elaborazione e integrazione esterna (vista di sistema).

- **Usabilità:** l'interfaccia deve permettere l'estensione fluida di funzionalità secondarie (es. filtri, esportazione) senza interrompere il flusso principale di creazione del viaggio.

- **Efficienza e accuratezza:** il calcolo dell'ottimizzazione deve essere tempestivo e basato su dati geografici reali forniti da terze parti.
- **Sicurezza:** garantire l'accesso protetto alle funzionalità di gestione profilo e la persistenza sicura dei percorsi salvati e dello storico.

1.3 Casi D'uso

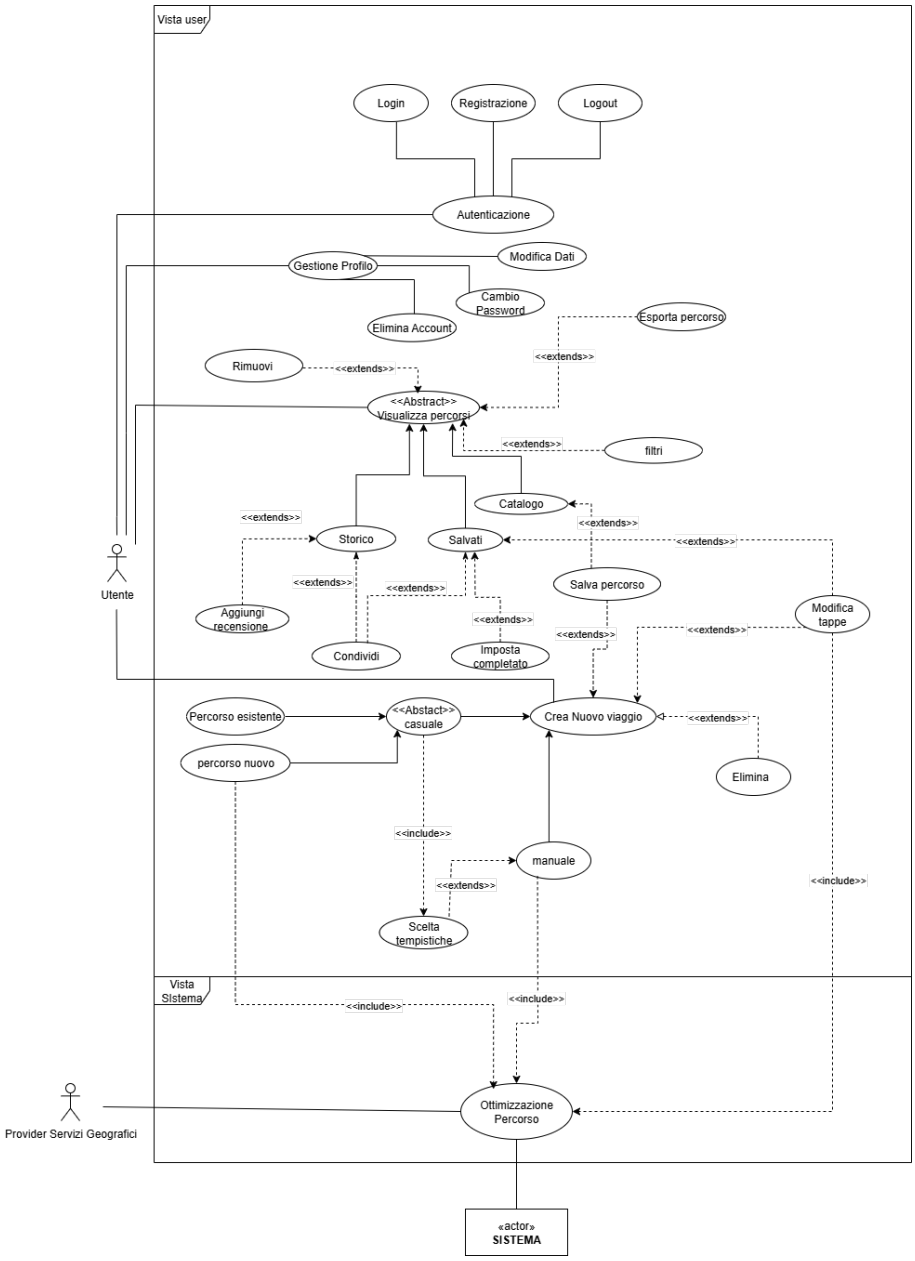

Figura 1.1: Casi d'uso

Di seguito sono riportati i casi d'uso relativi al sistema di gestione percorsi e viaggi.

- **UC1 - Autenticazione**
  - **UC1.1 - Registrazione:** un nuovo utente può creare un account personale fornendo i dati richiesti.

  - **UC1.2 - Login:** un utente registrato può accedere al proprio account inserendo le proprie credenziali per iniziare una sessione e usare le funzionalità del sistema.
  - **UC1.3 - Logout:** l'utente può terminare la propria sessione in modo sicuro.

- **UC2 - Gestione profilo**
  - **UC2.1 - Modifica dati:** l'utente può modificare i propri dati personali in caso di neccessità.
  - **UC2.2 - Elimina account:** l'utente può eliminare definitivamente il proprio account.

- **UC3 - Visualizza percorsi:** l'utente può visualizzare i dettagli di un percorso.
  - **UC3.1 - Catalogo:** è possibile visualizzare percorsi creati da altri utenti e inseriti in un catalogo.
  - **UC3.2 - Salvati:** è possibile visualizzare percorsi provenienti dall'elenco dei percorsi salvati.
  - **UC3.3 - Storico:** è possibile visualizzare percorsi provenienti dallo storico dei percorsi effettuati.

- **UC4 - Filtri ricerca:** l'utente può filtrare i percorsi in base a diversi criteri.

- **UC5 - Salva percorso:** l'utente può salvare un percorso nella sezione dei percorsi preferiti.

- **UC6 - Esporta percorso:** l'utente può esportare un percorso in un formato condivisibile.

- **UC7 - Rimuovi percorso salvato:** l'utente può rimuovere un percorso dai percorsi salvati.

- **UC8 - Condividi percorso:** l'utente può condividere un percorso con altri utenti.

- **UC9 - Aggiungi recensione:** l'utente può aggiungere una recensione a un percorso che è satao effettuato.

- **UC10 - Imposta percorso completato:** l'utente può contrassegnare un percorso come completato.

- **UC11 - Imposta punto di partenza:** l'utente seleziona il punto da cui far partire e finire il percorso.

- **UC12 - Crea nuovo viaggio:**
  - **UC12.1 - Manuale:** l'utente può far creare il proprio percorso scegliendo autonomamente le tappe da visitare.
  - **UC12.2 - Casuale:**
    - **UC12.2.1 - Percorso esistente:** l'utente fa scegliere al sistema in modo casuale un percorso già esistente nel catalogo.
    - **UC12.2.2 - Percorso nuovo:** l'utente fa generare al sistema in modo casuale un nuovo percorso.
- **UC13 - Scelta tempistiche:** l'utente può definire le tempistiche del viaggio.
- **UC14 - Ottimizzazione percorso:** il sistema ottimizza tramite un algoritmo specifico l'ordine delle tappe.
- **UC15 - Modifica tappe:** l'utente può modificare le tappe di un percorso.
- **UC16 - Elimina tappa:** l'utente può eliminare una tappa dal percorso.

1.4 Deployment Diagram

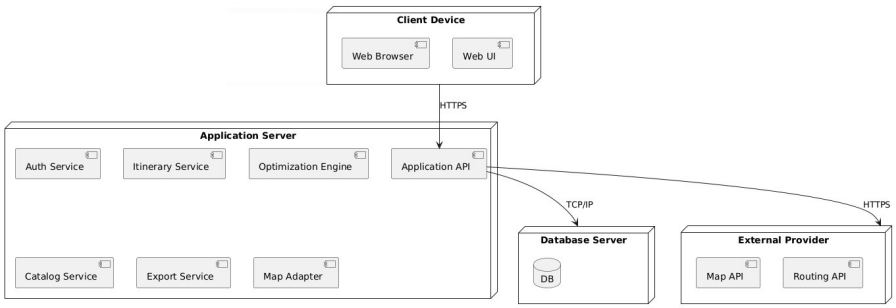

Figura 1.2: Diagramma di deployment

La Figura mostra l'organizzazione del sistema secondo un modello multilivello che separa l'interfaccia utente, i servizi applicativi e le integrazioni esterne.

- **Livello Client.** L'accesso al sistema avviene tramite un browser web. La Web UI comunica con il backend attraverso richieste `HTTPS` indirizzate all'Application API. Questo livello si occupa esclusivamente della presentazione e dell'interazione con l'utente finale.
- **Application Server.** Costituisce il cuore dell'applicazione e ospita diversi servizi interni:

  - **Auth Service**: gestisce autenticazione e autorizzazione;
  - **Itinerary Service**: gestisce la creazione e modifica degli itinerari;
  - **Optimization Engine**: calcola il percorso ottimale tra i monumenti selezionati;
  - **Catalog Service**: fornisce i dati relativi ai monumenti e ai punti di interesse;
  - **Export Service**: genera output esportabili;
  - **Map Adapter**: gestisce le comunicazioni con i servizi di mappe esterni.

  Tutti i servizi comunicano tramite l'Application API, che funge da punto di orchestrazione.

- **Database Server.** Ospita il database principale dell'applicazione e comunica con l'Application API. Gestisce dati persistenti come utenti, monumenti, itinerari e preferenze.

- **External Provider.** Il sistema si integra con servizi esterni per ottenere informazioni geografiche e di navigazione:

  - **Map API**: fornisce mappe, geocodifica e dati geografici;
  - **Routing API**: calcola distanze e tempi di percorrenza, supportando l'algoritmo di ottimizzazione.

1.5 Architettura del Sistema

Il sistema segue il pattern architetturale **Model-View-Controller (MVC)**, una scelta progettuale che permette di separare la logica di presentazione, la logica di ottimizzazione degli itinerari e la gestione dei dati. Tale separazione favorisce la modularità del codice, facilitando la manutenzione e l'estendibilità del software nel tempo.

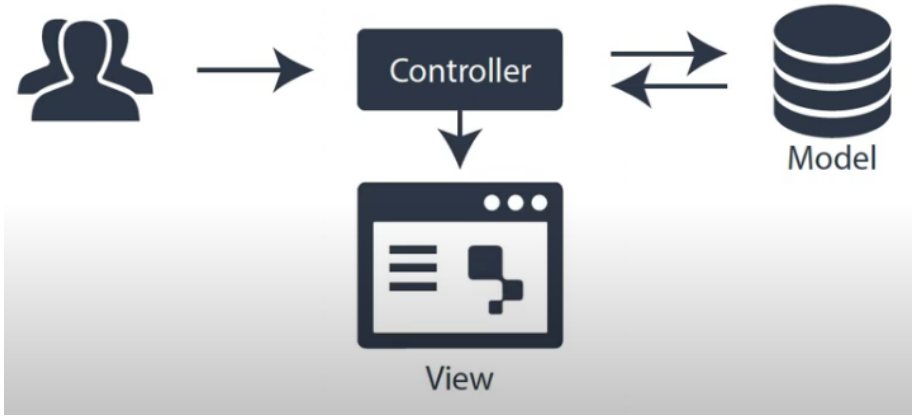

Figura 1.3: Archiettura MVC del Sistema

Model. Costituisce il nucleo logico e strutturale del sistema, responsabile della gestione dei dati, delle regole di dominio e dei meccanismi di persistenza dei dati.
In questo sistema:

- **Database NoSQL:** i dati sono memorizzati in MongoDB. La scelta di un database non relazionale è dettata dalla necessità di gestire con flessibilità i Points of Interest (POI) e i dati geografici. Per la gestione e la visualizzazione dei dati a livello amministrativo, viene utilizzata la GUI MongoDB Compass
- **Backend in Java:** il core del sistema è implementato utilizzando il framework Spring Boot, che orchestra i servizi attraverso un'architettura orientata ai componenti.
- **Integrazione geografica:**
  - **OpenStreetMap:** utilizzato come provider per le coordinate geografiche e i metadati dei POI.
  - **GraphHopper:** integrato come motore di routing per il calcolo del percorso ottimale tra i punti selezionati.
- **Data Access Layer:** l'interazione con il database avviene tramite Spring Data MongoDB.

View. È responsabile dell'interfaccia utente e della visualizzazione dei dati elaborati dal sistema:

- **React.js:** il frontend è sviluppato con la libreria React, utilizzando componenti funzionali e JSX per una UI reattiva. Lo sviluppo avviene tramite Visual Studio Code, ottimizzato per la gestione dei pacchetti e il debug del frontend.

- **Mapping e visualizzazione:** per la resa grafica delle mappe e dei percorsi viene utilizzata react-leaflet, permettendo di visualizzare i layer di OpenStreetMap direttamente nel browser.
- **Ambiente di esecuzione:** l'applicazione frontend viene gestita tramite Node.js, utilizzato per la gestione delle dipendenze e l'avvio del server di sviluppo locale.
- **Gestione API:** le chiamate asincrone verso il backend sono affidate ad Axios, garantendo uno scambio dati fluido tra client e server.

Controller. Funge da intermediario tra il Model e la View, gestendo le richieste dell'utente e ottimizzando la logica di interazione:

- **API:** il backend espone endpoint documentati tramite i controller di Spring Boot. Ogni richiesta HTTP inviata dal frontend tramite Axios viene gestita dal metodo specifico.
- **Ottimizzazione:** il controller riceve in input coordinate di partenza e di arrivo, interroga i servizi di routing e restituisce i risultati elaborati, in base al percorso ottimizzato.
- **Formato dati:** lo scambio di informazioni tra client e server avviene esclusivamente in formato JSON.